Name: Y.Veerendar Reddy **Reg.No:-** 192325102

Subject Code/Name: MLA0305-Reinforcement Learning

EXPERIMENT – 6

Monte Carlo Prediction and Control for Reinforcement Learning

Aim

To implement Monte Carlo Prediction and Control to estimate state values and determine an optimal policy from sampled episodes.

Procedure

1. Define the states, actions, and rewards.
2. Generate episodes by following an ε-greedy policy.
3. Calculate the return for each visited state.
4. Update the state-value estimates using the average returns.
5. Improve the policy based on the estimated values.
6. Repeat the process for several episodes.
7. Visualize the learned state values.

Code:

```
# ============================================================
# EXPERIMENT 6: MONTE CARLO PREDICTION AND CONTROL
# ============================================================

import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

states = ["Start", "Middle", "Goal"]
actions = ["Move", "Stay"]
gamma = 0.9
episodes = 500

# Rewards
```

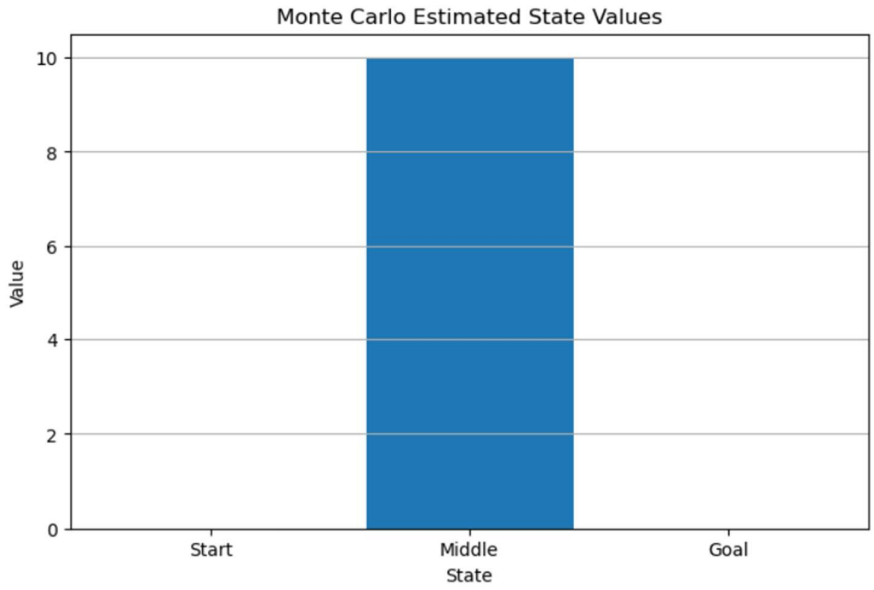
```
R = {
   ("Start", "Move"): 1, ("Start", "Stay"): -1,
   ("Middle", "Move"): 10, ("Middle", "Stay"): -1,
   ("Goal", "Move"): 0, ("Goal", "Stay"): 0
}

# Transition function
def next_state(s, a):
   if s == "Start":
      return "Middle" if a == "Move" else "Start"
   if s == "Middle":
      return "Goal" if a == "Move" else "Middle"
   return "Goal"

# Initialize values and policy
V = {s: 0.0 for s in states}
returns = {s: [] for s in states}

policy = {
   "Start": "Move",
   "Middle": "Move",
   "Goal": "Stay"
}

# Monte Carlo Control
for ep in range(episodes):

   state = "Start"
   episode = []

   # Generate episode
   for step in range(10):

      action = policy[state]
      reward = R[(state, action)]
```

```
        episode.append((state, reward))

        state = next_state(state, action)

        if state == "Goal":
            break

    # Calculate returns
    G = 0

    for state, reward in reversed(episode):
        G = reward + gamma * G

        if state not in [x[0] for x in episode[:-1]]:
            returns[state].append(G)
            V[state] = np.mean(returns[state])

# Display results
print("Monte Carlo State Values")
print("-" * 30)

for s in states:
    print(f"{s}: {V[s]:.3f}")

print("\nLearned Policy")
print("-" * 30)

for s in states:
    print(f"{s} -> {policy[s]}")

# Visualization
plt.figure(figsize=(8, 5))
plt.bar(states, [V[s] for s in states])
plt.title("Monte Carlo Estimated State Values")
plt.xlabel("State")
```

```
plt.ylabel("Value")
plt.grid(axis="y")
plt.show()
```

OUTPUT:-

```
Monte Carlo State Values
------------------------------
Start: 0.000
Middle: 10.000
Goal: 0.000

Learned Policy
------------------------------
Start -> Move
Middle -> Move
Goal -> Stay
```



Visualization

The bar graph shows the estimated value of each state after Monte Carlo sampling. A higher value indicates that the state provides a better expected future return under the learned policy.

Summary

Monte Carlo Prediction and Control were implemented by generating episodes and estimating state values from the observed returns. The estimated values were then visualized.

Result

Thus, Monte Carlo Prediction and Control were successfully implemented, and the estimated state values were obtained and visualized.